# Sprint 16: Introduction to Neural Networks

## Agenda

**1. Overview of Neural Networks**

- What are neural networks?
- Importance of Neural Networks
- Real-world applications

**2. History of Neural Networks**

**3. Components of a Neural Network**

- Neurons
- Layers
    - Input Layer
    - Hidden Layers
    - Output Layer
- Weights and Biases

**4. Neural Network Architectures**

- The Perceptron
- Feed-Forward Networks
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks (RNNs)
- Generative Adversarial Network (GAN)
- Summary of Architectures

**5. Building a Basic Neural Network**

- Defining the Model with **TensorFlow/Keras**
- Display the model architecture

**6. The Mathematics of Neural Networks**

- Weights, Biases, and Shifts
- Backpropagation

### 7. Feedforward Process

- Forward Propagation
- Mathematical Formula
- Non-Linearity

### 8. Activation Functions Overview

- Introduction to Activation Functions
- Common Activation Functions
    - Sigmoid Function
    - ReLU
    - Tanh

### 9. Loss Functions Overview

- What is a Loss Function?
- Common Loss Functions
    - Mean Squared Error
    - Binary Crossentropy

### 10. Training a Neural Network

- Dataset Creation
- Compiling the Model
- Training the Model

### 11. Model Overfitting and Regularization

- Bias and Variance in Overfitting
- Techniques
    - L1 and L2
    - Dropout

### 12. Evaluating the Model

- Evaluating Loss and Accuracy
- Visualizing Training Process

**Conclusion**

- Key Insights

# 1. Overview of Neural Networks

↳ 1 cell hidden

# 2. History of Neural Networks

↳ 1 cell hidden

# 3. Components of a Neural Network

The Neural Network architecture is made of individual units called **neurons** that mimic the biological behavior of the brain.

## Neurons

Artificial neurons, also known as perceptrons, are the fundamental building blocks of a neural network. They are inspired by biological neurons, which receive inputs from other neurons, process them, and produce outputs. In a neural network, an artificial neuron similarly receives inputs, processes them by applying a weighted sum and a bias, and then passes the result through an activation function.

## Neural Networks Layers

A basic neural network has interconnected artificial neurons in three layers:

1. **Input Layer**: Receives the initial data.

Information from the outside world enters the artificial neural network from this layer. Input nodes process the data, analyze or categorize it, and pass it on to the next layer.

2. **Hidden Layers**: Intermediate layers where computation is performed.

Take their input from the input layer or other hidden layers. Artificial neural networks

can have a large number of hidden layers. Each hidden layer analyzes the output from the previous layer, processes it further, and passes it on to the next layer.

3. **Output Layer**: Produces the final output.

Gives the final result of all the data processing by the artificial neural network. It can have single or multiple nodes. For instance, if we have a binary (yes/no) classification problem, the output layer will have one output node, which will give the result as 1 or 0. However, if we have a multi-class classification problem, the output layer might consist of more than one output node.

## Weights and Biases

These are parameters that the network learns during training. Weights adjust the strength of the connections between neurons, while biases shift the activation function.

1. **Weights** are the most critical parameters in a neural network as they determine the strength and direction of the relationship between inputs and neurons. The weight matrix controls how much influence each input has on the output. The values of weights are updated during the training process to minimize the error between the predicted output and the actual output.
2. **Bias** term is an additional parameter that allows the model to fit the data better by shifting the activation function. It helps the model to output a non-zero result even when all the inputs are zero. Without a bias term, the neural network would be constrained to pass through the origin (0,0), which could limit its ability to fit certain patterns in the data.

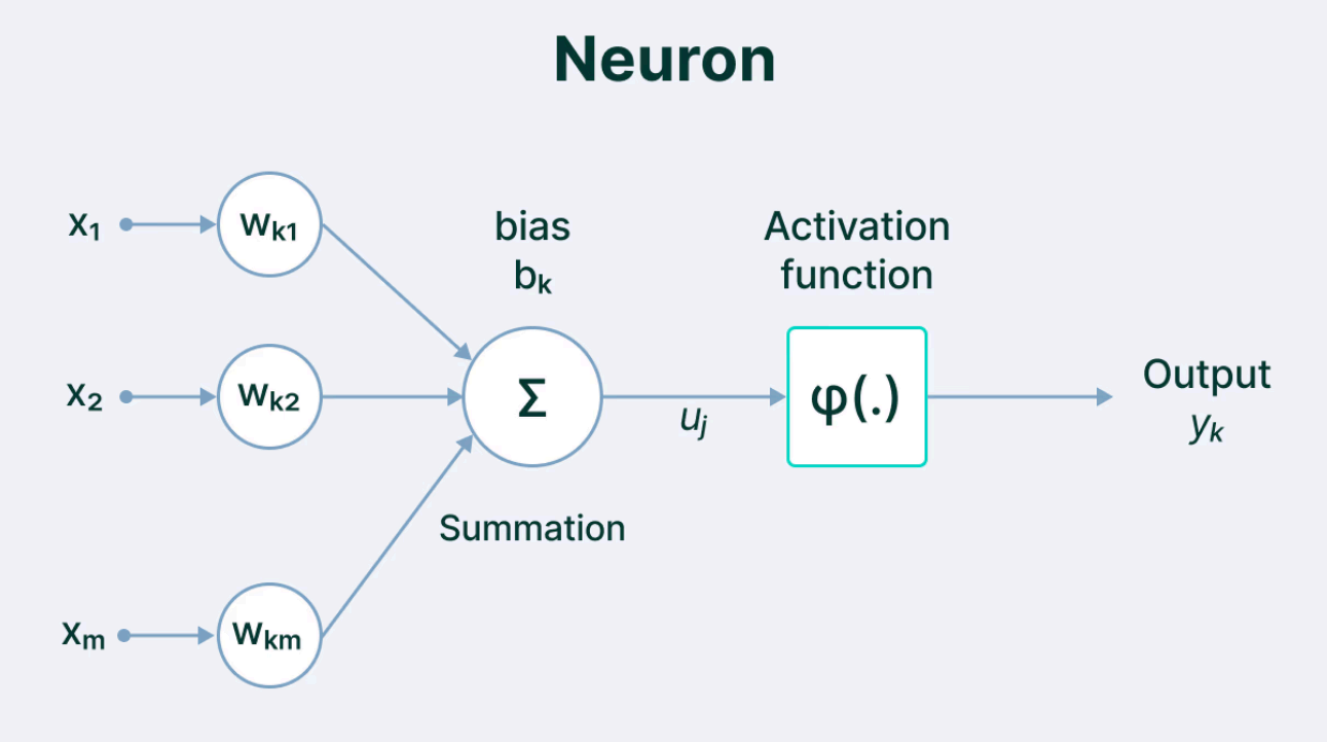
### Importance of Weights and Biases

- Weights control how much influence a particular feature has on the output, making them key to learning the underlying data patterns.
- Biases allow the network to adjust the output independently of the inputs, adding flexibility to the model.

### Activation Functions

An activation function in a neural network is a mathematical function applied to the output of a neuron to introduce non-linearity into the model. This non-linearity allows

the network to learn and model complex patterns in data, which would be impossible with only linear transformations.




## 4. Neural Network Architectures

- **The Perceptron**

A **Perceptron** is the most basic form of a Neural Network.

It processes multiple inputs, applies mathematical operations, and generates an output. The perceptron accepts a vector of real-valued inputs and computes a linear combination of these inputs, where each attribute is multiplied by a corresponding weight.

The weighted inputs are then summed to produce a single value, which is passed through an activation function.

Multiple perceptron units can be combined to construct more complex Artificial Neural Network architectures.

- **Feed-Forward Networks**

A **Perceptron** models the behavior of a single neuron, but what happens when multiple perceptrons are connected in layers? How does such a network learn?

This structure is known as a **multi-layer Neural Network**. As the name implies, information flows in a **forward direction**—from left to right—through the network.

During the **forward pass**, data enters through the **input layer**, moves sequentially through one or more **hidden layers**, and finally reaches the **output layer**. Unlike recurrent networks, **Feed-Forward Networks** have no loops or connections that send information back to previous layers.

The learning process in Feed-Forward Networks follows the same fundamental principles as a perceptron, enabling the model to learn and make predictions effectively.

- **Residual Networks (ResNet)**

When designing a neural network, a common question arises—**how many layers should the architecture have?**

A simple assumption might be that adding more hidden layers improves learning. While deeper networks can capture more complex features, they also face challenges —specifically, the **vanishing** and **exploding gradient problems**, which make training difficult.

**Residual Networks (ResNets)** address these issues by introducing an **alternate pathway** for data flow, enabling faster and more efficient training.

Unlike traditional feed-forward architectures, ResNets leverage a technique called **skip connections** (or **shortcuts**). These connections allow data to **bypass multiple layers** by directly carrying information from earlier layers to later layers.

The key idea is that a **deeper network** can be constructed from a **shallow network** by using **identity mappings**—copying weights from shallower layers. This approach prevents information loss, mitigates gradient issues, and simplifies optimization.

By enabling this data flow, ResNets have revolutionized deep learning, making it possible to train extremely **deep neural networks** effectively.

- **Convolutional Neural Networks (CNNs)**

**Convolutional Neural Networks (CNNs)** are a type of **Feed-Forward Neural**

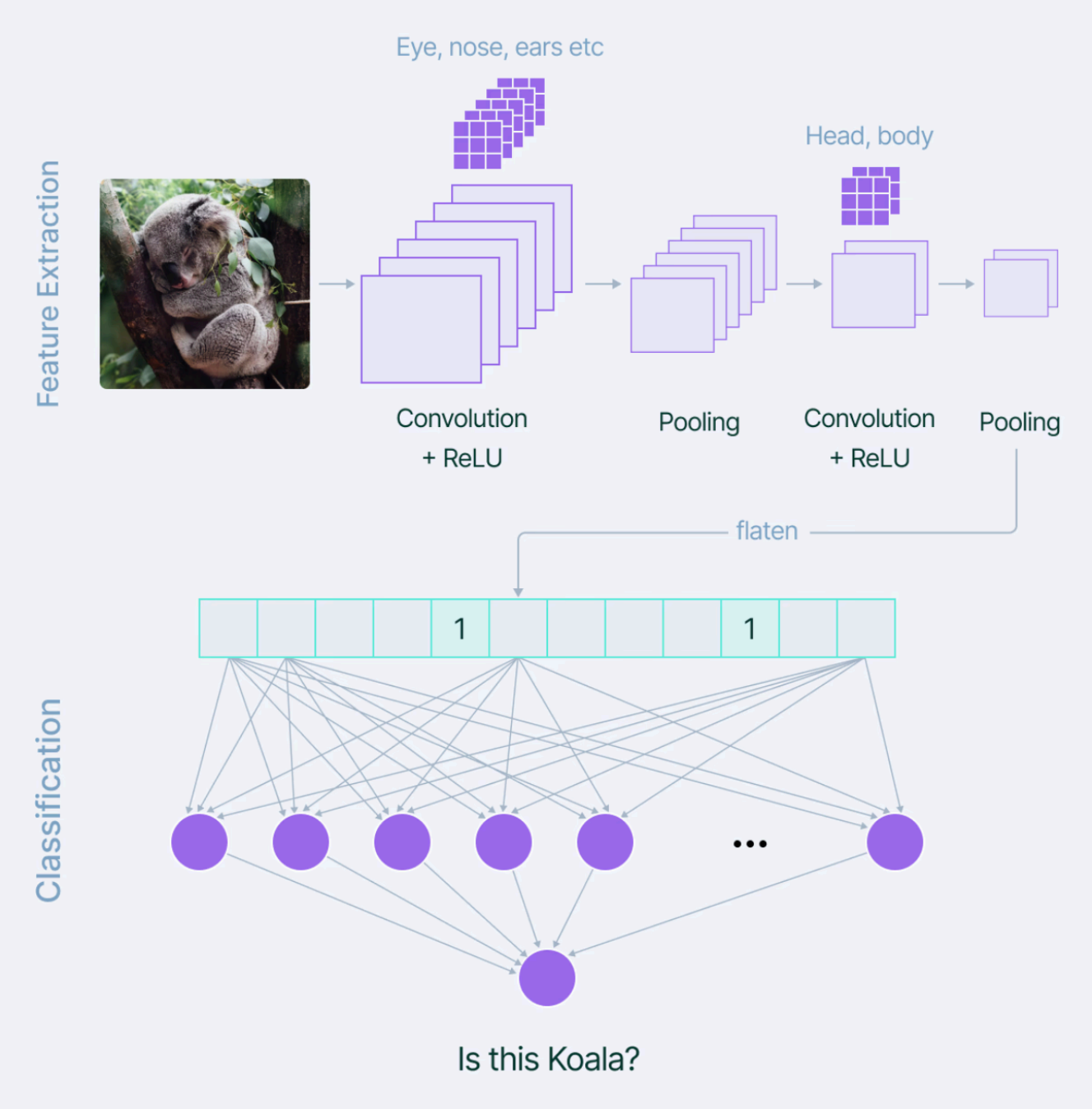
**Network** commonly used for tasks such as **image analysis**, **natural language processing**, and **complex image classification** problems.

CNNs are built upon multiple **hidden layers**, primarily **convolutional layers**, which serve as the foundation of these networks.

**Features** in image data, such as **edges**, **borders**, **shapes**, **textures**, and **objects**, represent fine details that CNNs aim to capture and analyze.

At a higher level, **convolutional layers** identify these patterns using **filters**. The **initial layers** focus on detecting basic patterns like edges and textures, while **deeper layers** handle more complex and abstract features.

As the network deepens, its ability to recognize intricate patterns improves. For instance, **early layers** may identify **simple shapes** and **edges**, while **later layers** can detect **specific objects** like **eyes**, **ears**, and even **animals** such as **cats** or **dogs**.




When adding a **convolutional layer** to a neural network, it is essential to define the **number of filters** to be used.

A **filter** can be visualized as a small matrix with a predefined number of **rows** and **columns**, which determines its dimensions.

Initially, the values in this **filter matrix** are set to **random numbers**. As the **convolutional layer** processes the input data, the filter moves across patches of the input matrix, performing mathematical operations to extract relevant features.

The output from the **convolutional layer** is typically passed through the **ReLU (Rectified Linear Unit)** activation function to introduce **non-linearity** into the model. The ReLU function modifies the feature map by replacing all **negative values** with **zero**, enabling the network to learn complex patterns.

An additional critical step in CNNs is **pooling**, which reduces the **computational complexity** and ensures the model is more **robust** to **distortions** and **variations** in the input data.

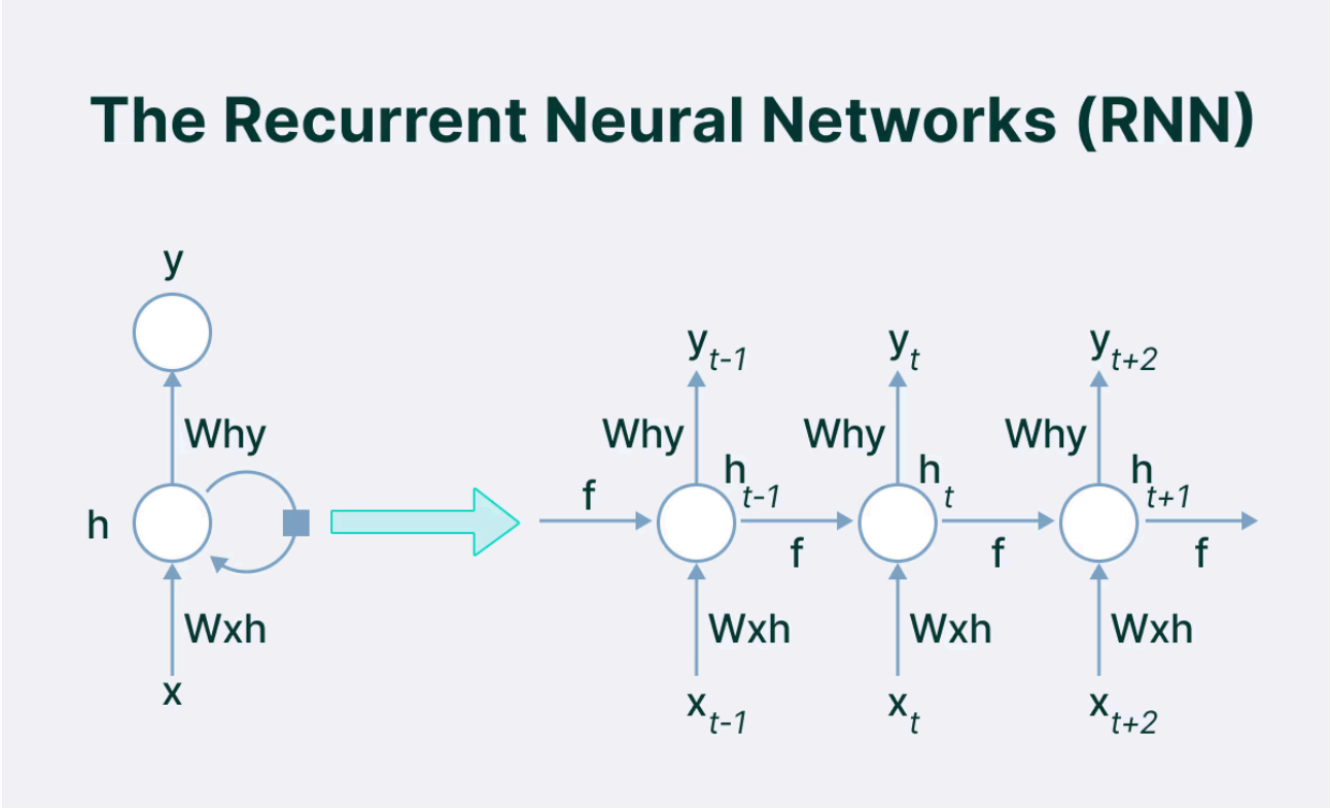
Finally, the **flattened feature matrix** produced by the convolutional and pooling layers is passed through a **fully connected dense layer**, which makes predictions based on the extracted features.

- **Recurrent Neural Networks (RNNs)**

Traditional deep learning architectures are designed to handle **fixed input sizes**, which can be a limitation in scenarios where the input size varies. Additionally, these models rely solely on the **current input** for decision-making, without the ability to **retain information** from previous inputs.

**Recurrent Neural Networks (RNNs)** are specifically designed to process **sequential data**, making them well-suited for tasks such as **natural language processing (NLP)**—including **sentiment analysis** and **spam detection**—as well as **time-series forecasting**, like **sales predictions** and **stock market analysis**.

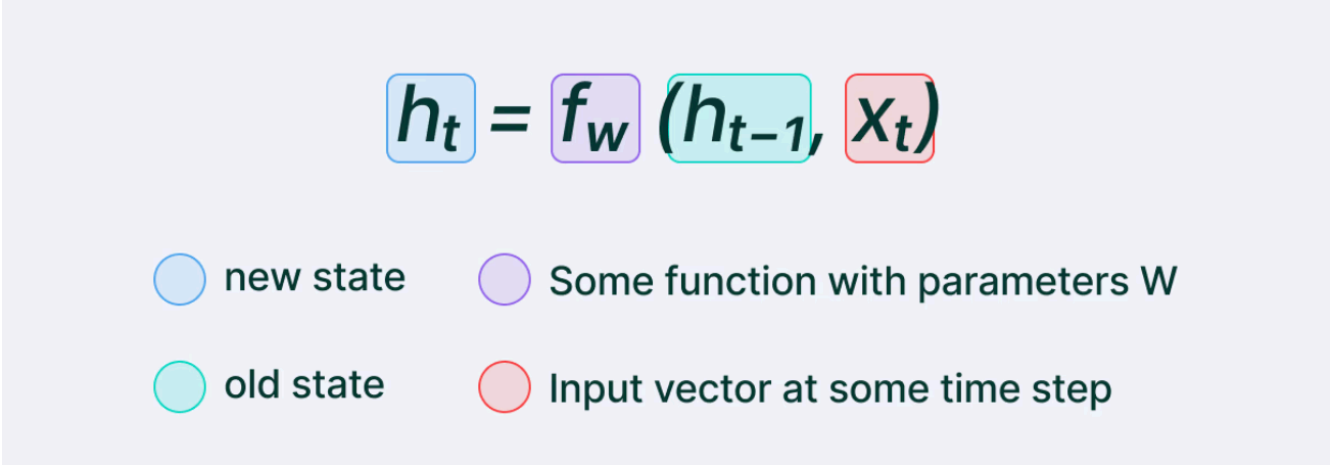
**Recurrent Neural Networks (RNNs)**
RNNs possess the unique ability to **store information** from prior inputs and utilize it for **future predictions**. This memory-like capability allows them to effectively learn patterns over time, enabling them to handle complex sequential data with ease.




The input to an **RNN** is provided as **sequential data**, which is processed step-by-step. The network maintains a **hidden internal state** that is updated each time it reads the next element in the input sequence.

This **hidden state** is then fed back into the model, enabling it to retain information about previous inputs. At each **timestamp**, the RNN generates an **output** based on the current input and its internal state.

The mathematical representation of this process is as follows:

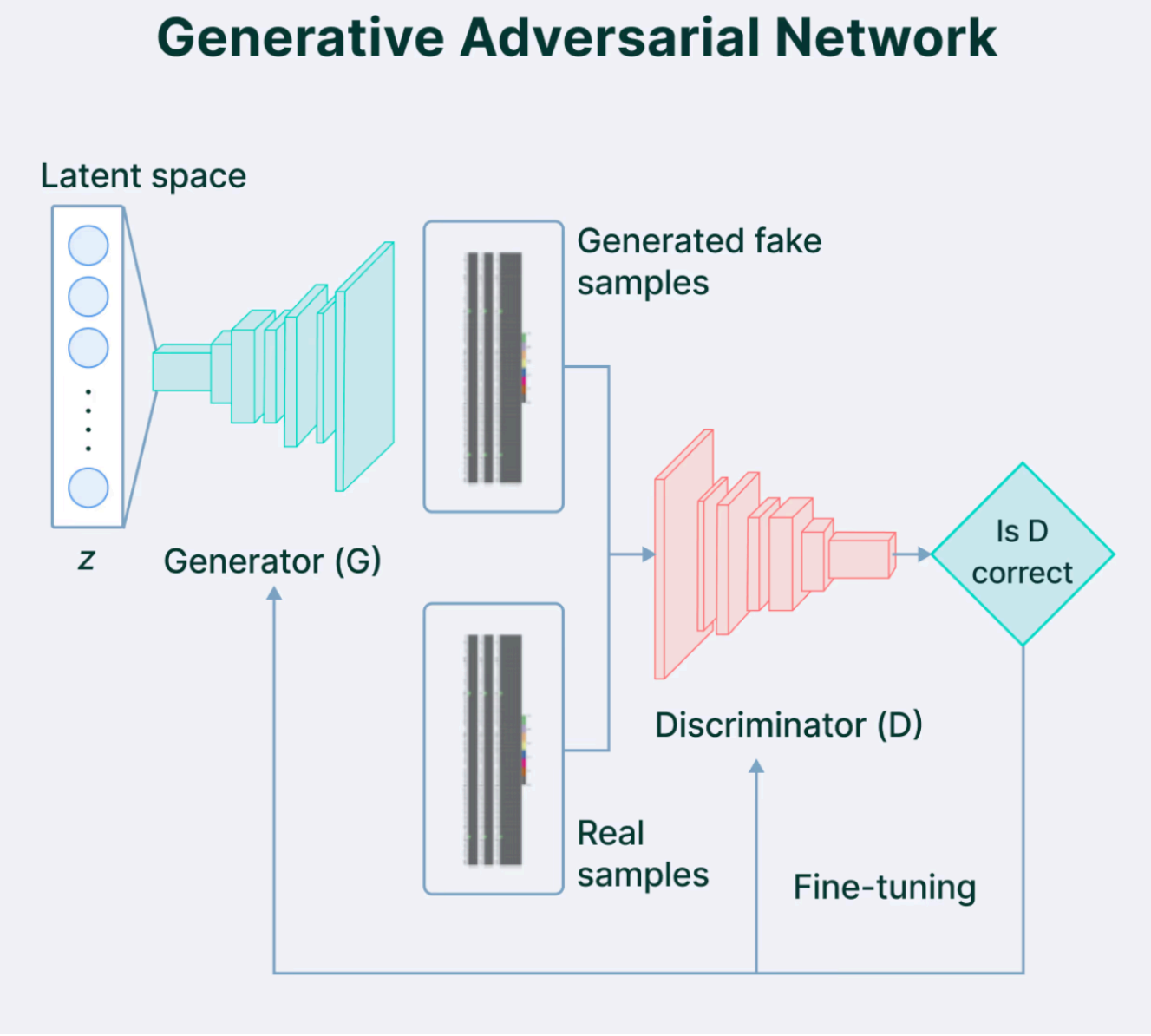



Note: We use the same function and parameters at every timestamp.

- **Generative Adversarial Network (GAN)**

**Generative modeling** is a branch of **unsupervised learning** that focuses on creating **new or synthetic data** by identifying patterns within the input dataset.

A **Generative Adversarial Network (GAN)** is a type of **generative model** designed to produce entirely **new synthetic data** by learning and mimicking patterns observed in the input data. It remains a highly **active area of research** in **artificial intelligence (AI)** due to its ability to generate realistic outputs.




### Generative Adversarial Networks (GANs)

GANs consist of two primary components—**a generator** and **a discriminator**—that operate in a **competitive framework**.

- The **generator** is responsible for producing **synthetic data** by learning patterns during training. It takes **random noise** as input and applies a series of **transformations** to generate data, such as images.
- The **discriminator** acts as a **critic**, possessing an understanding of the **problem domain** and distinguishing between **real** and **generated (fake)** data.

The **discriminator** evaluates the generated images and assigns a **probability score** between **0 and 1**, where **1 represents a real image** and **0 denotes a fake one**.

## Summary of NN Architectures

### 1. Convolutional Neural Networks (CNNs)

- **Purpose**: Primarily used for image-related tasks such as classification, object detection, and segmentation.
- **Key Features**:
  - Uses convolutional layers to extract spatial features from data.
  - Excellent for handling grid-like data (e.g., images with height, width, and channels).
  - Incorporates pooling layers to reduce dimensionality and computational cost.
- **Applications**:
  - Image recognition (e.g., identifying objects in photos).
  - Video analysis.
  - Medical imaging (e.g., tumor detection).

### 2. Recurrent Neural Networks (RNNs)

- **Purpose**: Designed for sequential data and time series.
- **Key Features**:
  - Maintains a hidden state to capture dependencies over time.
  - Processes input sequentially, making it ideal for temporal or sequential patterns.

    - Variants like LSTMs (Long Short-Term Memory) and GRUs (Gated Recurrent Units) address issues like vanishing gradients.

- **Applications**:
    - Natural Language Processing (NLP) tasks like language translation and sentiment analysis.
    - Time series prediction (e.g., stock prices or weather forecasting).
    - Speech recognition.

## 3. Generative Adversarial Networks (GANs)

- **Purpose**: Used for generative tasks to create new data samples that resemble a given dataset.
- **Key Features**:
    - Composed of two networks: a **Generator** and a **Discriminator**.
    - The Generator creates data, and the Discriminator evaluates it. Both are trained adversarially to improve data generation quality.
    - Effective in learning high-dimensional data distributions.
- **Applications**:
    - Image generation (e.g., creating realistic faces or art).
    - Data augmentation.
    - Super-resolution (enhancing image quality).

## Summary of Key Differences

| Feature | CNN | RNN | GAN |
|---|---|---|---|
| **Data Type** | Spatial (e.g., images) | Sequential (e.g., text, time) | High-dimensional (e.g., images) |
| **Structure** | Convolutional layers | Recurrent connections | Generator-Discriminator pair |
| **Applications** | Image-related tasks | Sequence prediction tasks | Data generation tasks |

Each architecture excels in its domain and is tailored to the specific challenges of the data it processes.

# 5. Creating a Basic Neural Network with TensorFlow/Keras

In this section, we'll use **TensorFlow** and **Keras** to build a simple neural network.

TensorFlow is an open-source library developed by Google, and Keras is a high-level API that simplifies building and training neural networks.

## Installing TensorFlow

If you're using Google Colab, you can skip this part, as TensorFlow and Keras are **pre-installed**.

However, if you're running this locally, ensure you have the following libraries:

```
!pip install tensorflow
```

Show hidden output

## Importing Libraries

Let's import the necessary libraries:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
```

## Structure of a Simple Neural Network

We'll first create a simple neural network using tensorflow and keras.

```
# Define a simple neural network
model = Sequential()

# Adding input layer with 3 neurons in the input layer
model.add(Dense(units=3, input_shape=(3,), activation='relu'))

# Adding one hidden layer with 5 neurons
model.add(Dense(units=5, activation='relu'))

# Output layer with 1 neuron for binary classification (sigmoid activa
model.add(Dense(units=1, activation='sigmoid'))

# Display the model architecture
model.summary()
```

## 6. The Mathematics of Neural Networks

At its core, a neural network computes a weighted sum of inputs, applies an activation function, and then passes the result to the next layer. The process repeats across multiple layers until the final output is produced.

The core operations include:

**Weighted Sum:**

$$Z = \sum_{i=1}^{n} w_i x_i + b$$

Where $w_i$ are the weights, $x_i$ are the input features, and $b$ is the bias term.

**Backpropagation:**

Neural networks learn by minimizing the error between the predicted output and the actual output. This is achieved through backpropagation, where the error is propagated backward through the network, and the weights are adjusted using gradient descent.

**Gradient Descent Formula:**

$$w_i = w_i - \alpha \frac{\partial J(w)}{\partial w_i}$$

Where $J(w)$ is the cost function, $\alpha$ is the learning rate, and $\frac{\partial J(w)}{\partial w_i}$ is the gradient of the cost function with respect to the weight $w_i$.

# 7. Feedforward Process

In a neural network, the **feedforward process** (also known as forward propagation) is the mechanism through which inputs move through the network from the input layer to the output layer. Each neuron processes the inputs it receives by performing a series of mathematical operations. These operations involve applying weights to the

inputs, summing them, adding a bias term, and passing the result through an activation function to produce the final output.

### Mathematical Process in Forward Propagation

1. **Inputs**: The network takes in a vector of inputs, `x = (x1, x2, ..., xn)`, where (n) is the number of features in the data.
2. **Weights and Biases**: Each neuron in a layer is associated with a set of **weights** `w = (w1, w2, ..., wn)` corresponding to the inputs, and a **bias** (b), which helps adjust the output independently of the input values.
3. **Weighted Sum**: The neuron computes the **weighted sum** of the inputs:

   `z = w1 * x1 + w2 * x2 + ... + wn * xn + b`

   In matrix form, for a layer of multiple neurons, this can be written as:

   `z = W * x + b`

   where:

   - **W** is the weight matrix,
   - **x** is the input vector,
   - **b** is the bias vector.

### Illustration of Forward Propagation

Consider a simple network with 2 input neurons, 1 hidden layer with 2 neurons, and 1 output neuron. The forward propagation steps look like this:

- **Input Layer**: `[x1, x2]`
- **Hidden Layer:** Each neuron computes its weighted sum and applies an activation function (e.g., ReLU).
- **Output Layer**: The final output is produced by applying a sigmoid activation to the result from the hidden layer.

## Importance of Non-Linearity

One of the key reasons neural networks are powerful is because of the **non-linearity** introduced by activation functions. Without non-linearity, the network would behave like a simple linear model, no matter how many layers it has. A linear combination of

linear functions is still linear, which limits the network's ability to learn complex patterns from data.

**Why Non-Linearity is Important**

- **Linearly Separable Data**: For problems where the decision boundary is a straight line (or plane in higher dimensions), linear models like logistic regression may suffice. However, most real-world data is **not linearly separable**, meaning a simple linear decision boundary cannot separate the classes.
- **Complex Patterns**: Non-linear activation functions (e.g., ReLU, Sigmoid) allow the network to model complex, non-linear relationships in the data, enabling it to capture intricate patterns, curves, and decision boundaries.

# 8. Activation Functions Overview

## Role of Activation Functions in Neural Networks

In neural networks, activation functions are essential components that introduce non-linearity into the model. Without activation functions, the neural network would behave like a linear model regardless of how many layers it has, limiting its ability to model complex patterns and relationships.

- **Linear Models**: A linear model makes predictions based solely on a linear combination of the input features. These models are only effective when the relationship between the inputs and outputs is linear. However, most real-world problems involve non-linear relationships, making linear models insufficient for capturing the full complexity of the data.
- **Non-linear Functions**: Activation functions introduce non-linearity, which allows the neural network to learn and model more complex relationships. With non-linear activation functions, the network can create intricate decision boundaries and represent more abstract features across layers. This makes it possible for neural networks to solve tasks like image classification, natural language processing, and more.

In essence, the activation function determines whether a neuron should be activated (i.e., produce an output) based on the weighted input. It plays a crucial role in deciding the final output of a neural network.

## Common Activation Functions

There are several activation functions used in neural networks, each with its own properties. Let's explore the most commonly used ones:

**Sigmoid Activation Function**

The sigmoid function squashes the output to a range between 0 and 1.

**Sigmoid Formula:**

$$\sigma(Z) = \frac{1}{1 + e^{-z}}$$

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Test sigmoid function
x = np.linspace(-10, 10, 100)
y = sigmoid(x)

plt.plot(x, y)
plt.title('Sigmoid Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.show()
```

Show hidden output

### ReLU (Rectified Linear Unit)

ReLU is the most common activation function. It returns the input if it is positive; otherwise, it returns zero.

**ReLU Formula:**

$$\text{ReLU}(Z) = \max(0, Z)$$

```
def relu(x):
    return np.maximum(0, x)

# Test ReLU function
x = np.linspace(-10, 10, 100)
y = relu(x)

plt.plot(x, y)
plt.title('ReLU Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.show()
```

Show hidden output

### Tanh Activation Function

The tanh function squashes the output to a range between -1 and 1.

**Tanh Formula:**

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

The Tanh Activation function is a popular choice in RNNs due to its suitability for processing sequential data, such as in speech recognition and time series analysis.

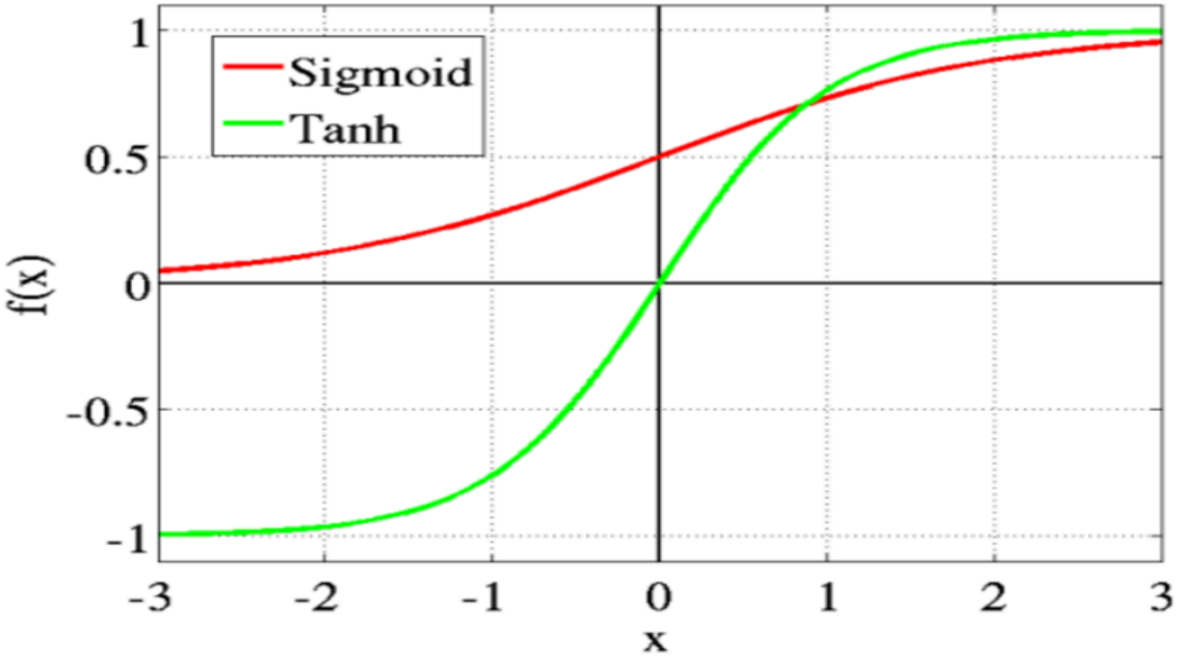



Fig: tanh v/s Logistic Sigmoid

```
def tanh(x):
    return np.tanh(x)

# Test Tanh function
x = np.linspace(-10, 10, 100)
y = tanh(x)

plt.plot(x, y)
plt.title('Tanh Activation Function')
plt.xlabel('Input')
plt.ylabel('Output')
plt.grid(True)
plt.show()
```

Show hidden output

# 9. Loss Functions Overview

# What is a Loss Function?

In machine learning, a loss function is a mathematical function that measures how well a model's predictions align with the actual target values. It quantifies the error between the predicted output of the neural network and the true output (ground truth). The goal of training a neural network is to minimize this error or "loss."

The loss function provides a feedback signal for the network to adjust its parameters (weights and biases) during the training process. The network learns by reducing the loss through an optimization algorithm like Gradient Descent or its variants. In every iteration of training (epoch), the model calculates the loss, computes the gradients, and updates the weights to minimize the loss function.

**Why Loss Functions Matter**

- **Model Optimization**: Loss functions guide the optimization process by providing a scalar value that the algorithm aims to minimize.
- **Performance Measurement**: The lower the loss, the better the model is performing on the given task. High loss indicates that the model is making poor predictions.

The choice of a loss function depends on the type of machine learning task (e.g., regression or classification) and the specific problem you are solving.

## Common Loss Functions

Let's explore some of the most commonly used loss functions in neural networks, depending on the problem type (regression or classification).

**Mean Squared Error (MSE)**

This is commonly used in regression problems. It calculates the average of the squares of the errors between predicted and true values.

```
def mean_squared_error(y_true, y_pred):
    return np.mean(np.square(y_true - y_pred))

# Example usage
y_true = np.array([1, 0, 1, 1])
y_pred = np.array([0.9, 0.2, 0.8, 0.7])
loss = mean_squared_error(y_true, y_pred)
print(f'Mean Squared Error: {loss}')
```

```
Mean Squared Error: 0.045
```

### Binary Crossentropy

This is used in binary classification problems. It measures the performance of a classification model whose output is a probability between 0 and 1.

```
def binary_crossentropy(y_true, y_pred):
    epsilon = 1e-15  # To avoid log(0)
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1

# Example usage
y_true = np.array([1, 0, 1, 1])
y_pred = np.array([0.9, 0.2, 0.8, 0.7])
loss = binary_crossentropy(y_true, y_pred)
print(f'Binary Crossentropy: {loss}')
```

```
Binary Crossentropy: 0.22708064055624455
```

## 10. Training a Neural Network

The training process involves feeding the training data into the model, calculating the loss, and updating the weights.

- **Epochs**: Specify the number of epochs, which is the number of times the entire training dataset will be passed through the model.
- **Batch Size**: Choose a batch size, which determines the number of samples processed before the model updates its weights.
- **Fitting the Model**: Use the `fit()` function to train the model on the training data and validate it on the validation data.

We'll train the model on a simple synthetic dataset. The goal is to classify data points into two categories.

```
# Create a simple dataset
from sklearn.model_selection import train_test_split

# Generate some synthetic data
data = np.random.randn(1000, 3)  # 1000 samples, 3 features
labels = (np.sum(data, axis=1) > 0).astype(int)  # Binary classificat

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(data, labels, tes

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=[

# Train the model
history = model.fit(X_train, y_train, epochs=10, validation_data=(X_t
```

Show hidden output

## 11. Model Overfitting and Regularization

For a machine learning model to be useful, it must **generalize well**—accurately predicting unseen data beyond its training set.

**Overfitting** is a common issue that hinders generalization. It happens when a model focuses too much on patterns in the training data, leading to poor performance on new data.

Think of a student who memorizes textbooks instead of understanding concepts. They excel at familiar questions but struggle with new problems—just like an overfitted model.

To address overfitting, **regularization** techniques are used during training. Before exploring these techniques, it's important to first understand the concepts of **bias** and **variance**.

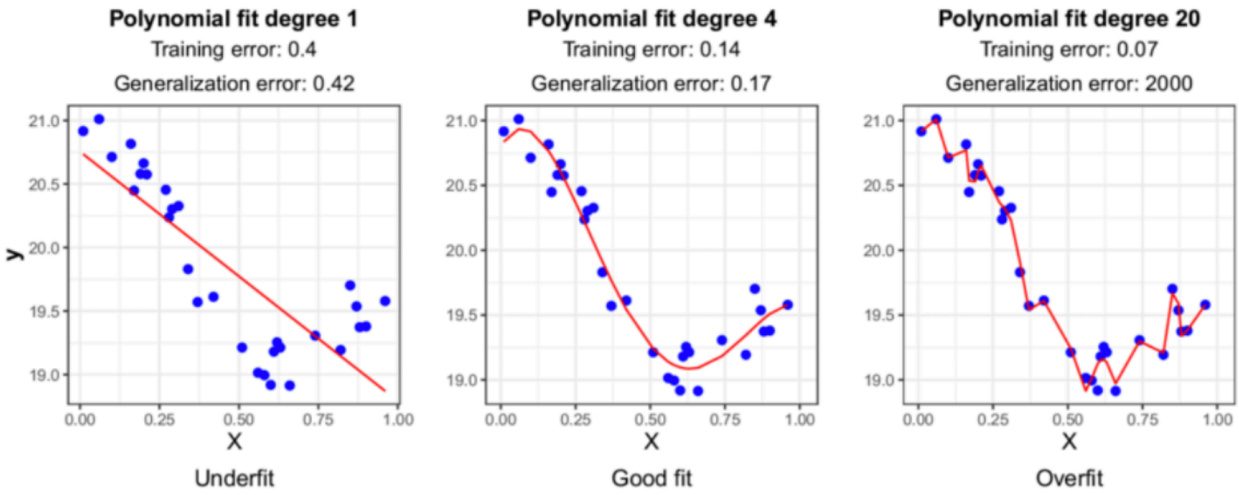
## Bias and Variance in Overfitting

**Bias** and **variance** are key concepts in machine learning that describe a model's behavior during training:

- **Bias** measures the difference between the model's predictions and the actual values. High bias indicates the model is too **simple** to capture patterns in the data.
- **Variance** reflects how much the model's predictions change with different training datasets. High variance means the model is **too sensitive** to small fluctuations, leading to **overfitting**.

These concepts relate to **underfitting** and **overfitting**:

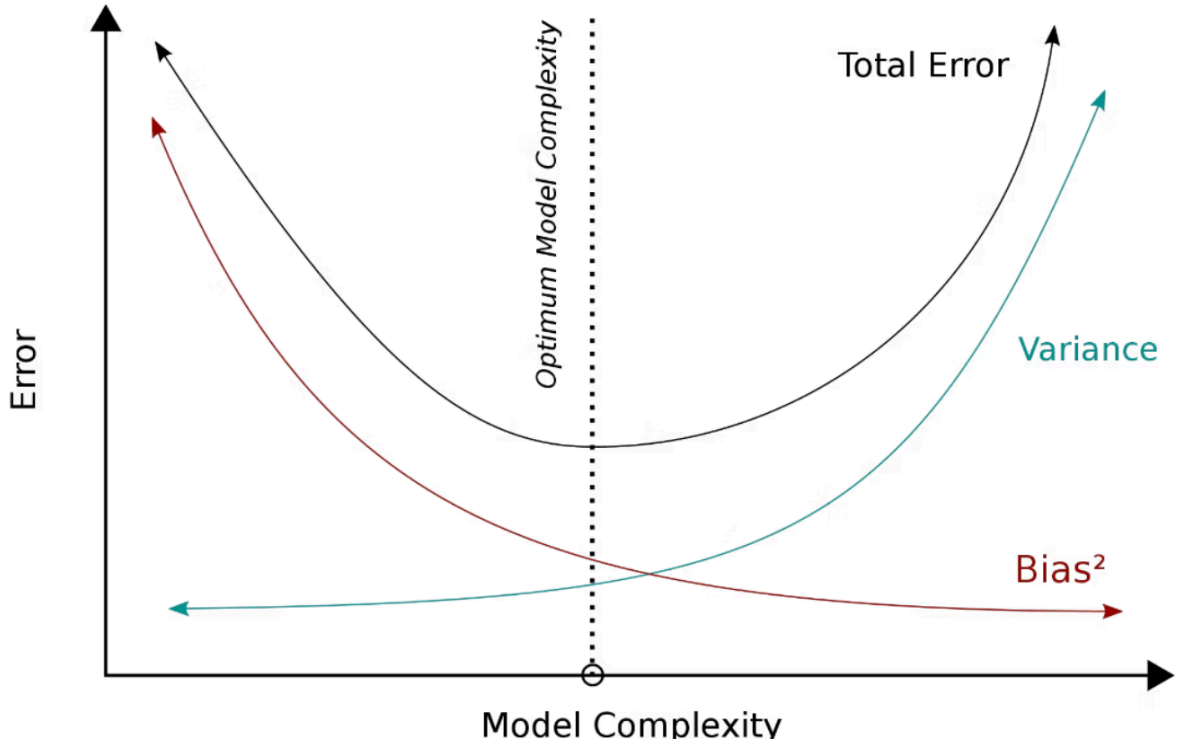
- **Underfitting** happens when a model with **high bias** and **low variance** fails to capture patterns, resulting in poor performance.
- To fix underfitting, the model's **complexity** can be increased, such as adding more layers to a neural network.




Piucture: Underfitting vs Good fit vs Overfitting.

**Overfitting** occurs when a model closely follows the training data, resulting in **poor generalization** to new data. It has **low bias** and **high variance**. To prevent this, we apply **regularization techniques**, which we'll discuss next.

Ideally, a model should balance **bias and variance** for optimal performance. However, practical challenges like **noisy data**, **limited data**, and **computational limits** make this difficult. The goal is to find a **trade-off** where the model performs well on both **training** and **test data**.




Picture: Bias and variance trade-off

# Different Kinds of Regularization Techniques

Several **regularization techniques** can enhance a model's ability to **generalize** to unseen data. In this section, we'll cover some of the most commonly used methods, including **L1 and L2 regularization**, **Elastic Net**, **Early Stopping**, **Dropout**, **Batch Normalization**, and **Data Augmentation**. We'll begin with **L1 and L2 regularization**.

## L1 and L2 Regularization

To understand **L1 and L2 regularization**, let's first review how **neural network weights** are optimized during training.

In each **epoch** or training step, the model's **loss function** is calculated based on the task:

- For **classification**, common loss functions include **binary** or **categorical cross-entropy**.
- For **regression**, the **mean squared error (MSE)** is often used.

$$Loss = Error(y, \hat{y})$$

Picture: Compute the loss function of our model

Next, the neural network optimizes its parameters using a **gradient descent algorithm**. In this process, the **partial derivative** of the **loss function** with respect to each **weight** is calculated. The weights are then updated by subtracting the **product** of the derivative and the **learning rate (λ)**, which is predefined.

$$w_{\text{new}} = w - \lambda \frac{\partial Error}{\partial w}$$

Picture: Perform gradient descent algorithm

**L1 and L2 regularization** add a **penalty term** to the **loss function** to discourage large weights, pushing them closer to **zero**. This simplifies the neural network, reducing its **complexity** and lowering the risk of **overfitting**.

Overfitting often occurs when a model is **too complex** for the training data. By shrinking the weights, regularization reduces **variance**, improving generalization.

**The differences between L1 and L2 regularization:**

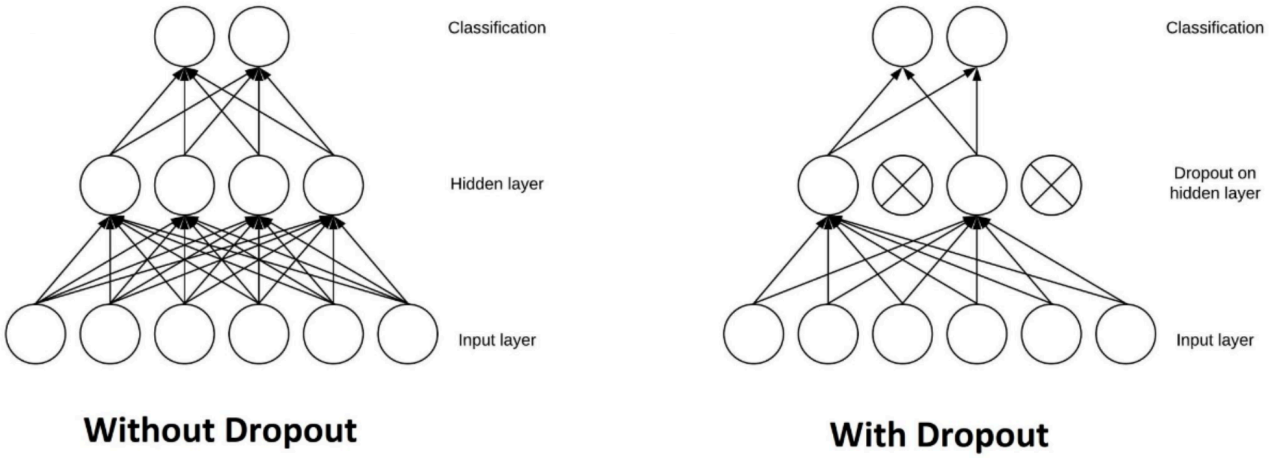
- L1 regularization penalizes the sum of absolute values of the weights, whereas L2 regularization penalizes the sum of squares of the weights.
- The L1 regularization solution is sparse. The L2 regularization solution is non-sparse.
- L2 regularization doesn't perform feature selection, since weights are only reduced to values near 0 instead of 0. L1 regularization has built-in feature selection.

- L1 regularization is robust to outliers, L2 regularization is not.

## Dropout

**Dropout** is a **regularization technique** used in neural networks to prevent **overfitting** by **randomly deactivating neurons** during training. When a neuron is dropped, its **weights are not updated** during **backpropagation**.

To apply Dropout, a **dropout probability** is set for each layer. For example, with a **50% dropout rate**, each neuron in the layer has a **50% chance** of being dropped during training. This randomness varies across **training batches**, resulting in slightly different **network architectures** within the same iteration, promoting **robustness** and reducing **overfitting**.




Picture: Architecture of a neural network before and after dropout.

**Dropout** may seem **destructive**, but it effectively improves a model's **generalization** by simplifying its structure. Overfitting often results from **complex models**, and randomly dropping neurons reduces this complexity.

When neurons are dropped, others must **adjust their weights** to compensate, making the model **less dependent** on specific neurons and enhancing **generalization**.

During **inference**, neurons are **not dropped**. Instead, outputs are **scaled down** by $1 / (1 - p)$, where $p$ is the dropout rate, ensuring consistent behavior.

# 12. Evaluating the Model

Once a neural network model is trained, it is crucial to evaluate its performance to understand how well it generalizes to unseen data. Evaluation helps determine whether the model overfits, underfits, or performs well on both the training and test datasets. In this section, we will cover key evaluation metrics, methods, and visualizations used to assess model performance.

## Importance of Model Evaluation

Evaluation is essential to:

- Assess Generalization: Check how well the model performs on new, unseen data.
- Avoid Overfitting/Underfitting: Ensure the model doesn't memorize the training data (overfitting) or fail to capture important patterns (underfitting).
- Choose the Right Model: Compare different models to find the one that offers the best performance for the problem at hand.

We begin by evaluating the trained model on the test dataset using the `evaluate()` function.

```
# Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f'Test Loss: {test_loss}')
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')
```

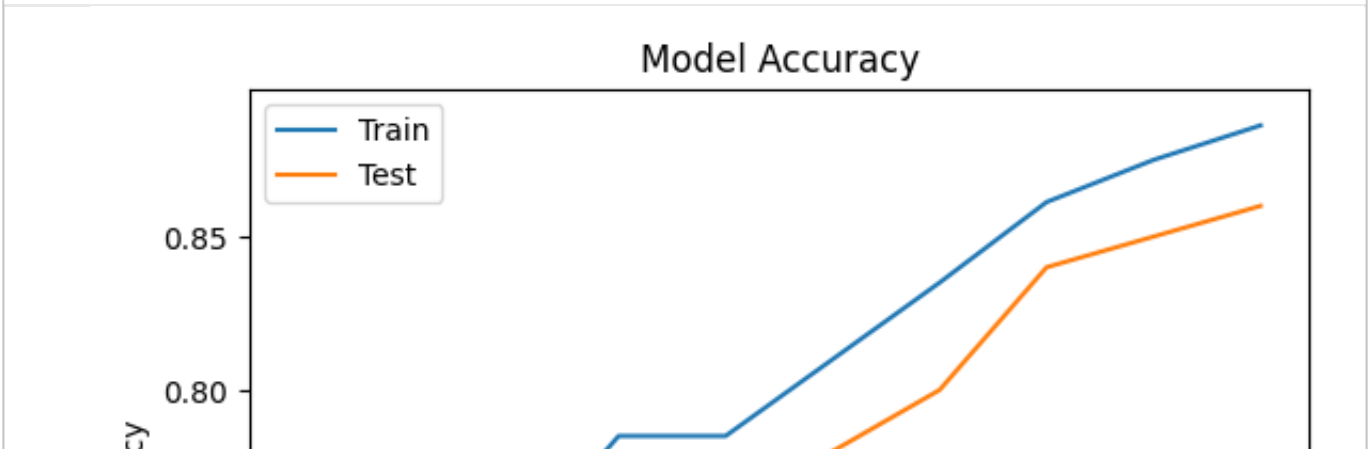
```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8557 - loss: 0.4373
Test Loss: 0.44484493136405945
Test Accuracy: 86.00%
```

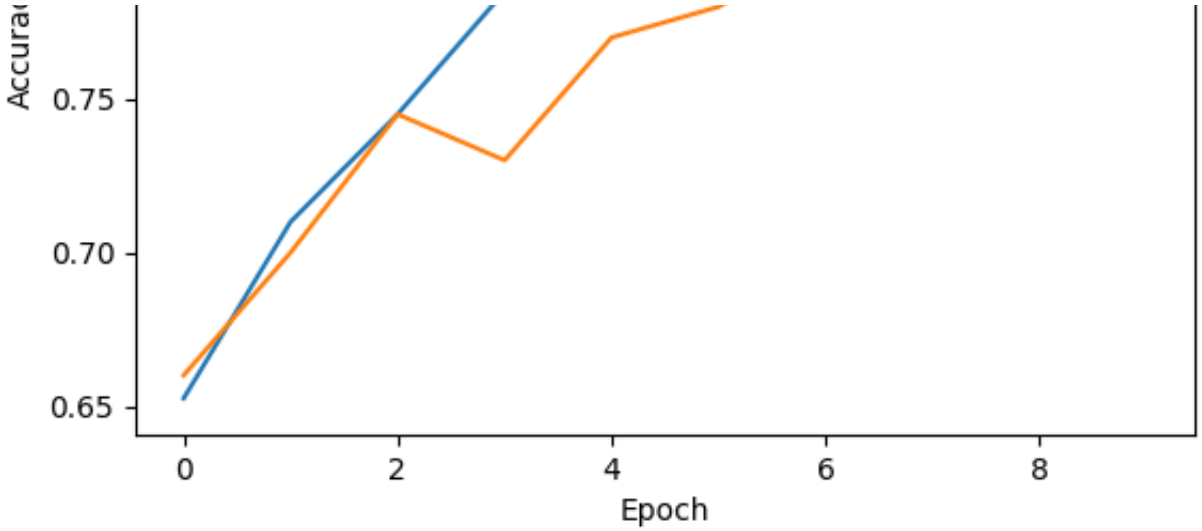
## Visualizations for Model Evaluation

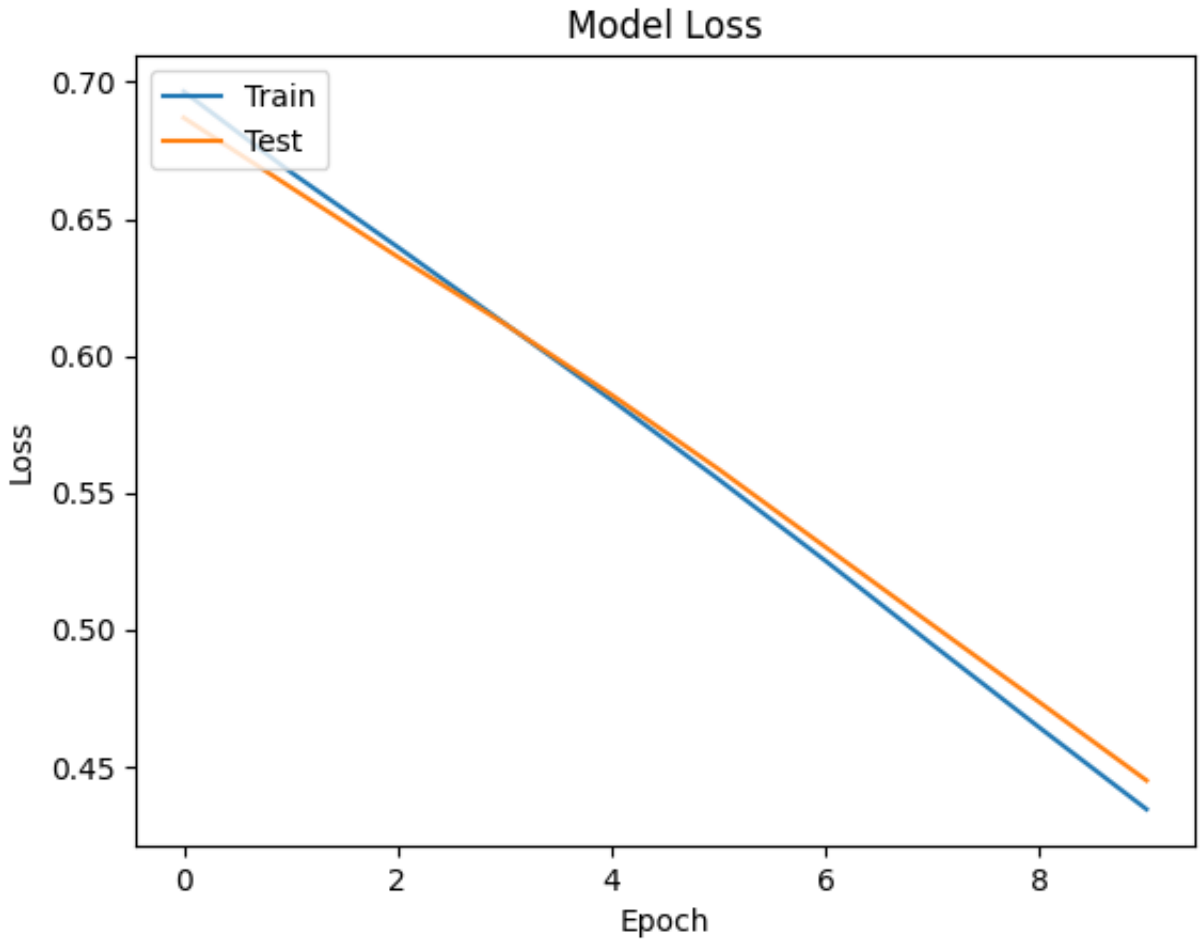
- **Confusion Matrix**: Visualizes the number of correct and incorrect predictions across classes.
- **ROC Curve and AUC (Area Under the Curve)**: Used to evaluate binary classifiers by plotting the true positive rate (recall) against the false positive rate at various threshold levels.
- **Precision-Recall Curve**: A plot of precision against recall for different thresholds.
- **Learning Curves**: Plot training and validation accuracy or loss over epochs to help diagnose overfitting or underfitting.

```
# Plot training & validation accuracy values
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()

# Plot training & validation loss values
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```




0.75
0.70
0.65
0
2
4
6
8
Epoch


Model Loss
Train
Test
Loss
0.70
0.65
0.60
0.55
0.50
0.45
0
2
4
6
8
Epoch

# Conclusion

In this Colab, we explored the basics of neural networks, covering their structure, the role of activation functions, and the importance of loss functions. We saw how layers of neurons process data, and how activation functions introduce non-linearity, enabling neural networks to model complex patterns. Loss functions guide the training process, helping the model improve its predictions.

**Key Insights:**

- **Neural networks** are powerful tools for solving a wide range of tasks, from classification to regression, thanks to their ability to learn from data.
- **Activation functions** introduce non-linearity, allowing networks to capture complex patterns.
- **Loss functions** and the training process are essential for guiding the model to improve its accuracy.

Each **neural network architecture** has its own strengths and limitations.

- **Feed-Forward Neural Networks** are widely used for **classification** and **regression** tasks involving **simple structured data**.
- **Recurrent Neural Networks (RNNs)** excel at handling **sequential data** like **text, audio, and video** due to their ability to **retain information** over time.
- For **complex data** such as **images**, **Convolutional Neural Networks (CNNs)** are effective for **classification**, while **Generative Adversarial Networks (GANs)** are ideal for **image generation** and **style transfer** tasks.

**Overfitting** occurs when a machine learning model fits too closely to the training data, resulting in poor performance on unseen data. **Regularization** is a technique used during training to **prevent overfitting**.

Common methods include **L1 and L2 regularization**, **Elastic Net**, **Early Stopping**, **Dropout**, **Batch Normalization**, and **Data Augmentation**. While these approaches differ, their shared goal is to **reduce variance** and **increase bias**, helping the model achieve a **balance** in the **bias-variance trade-off**.